Rotaract Club of Xiamen University Malaysia | Rotary Club of Kuala Lumpur Dynamics | Rotary Club of Tun Razak Exchange

SDC Hackathon

Technical Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club

Submission Deadline: 25 October 2026

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

Instagram: @sdc_xmumhack ***Email:*** sdc.xmumhack@gmail.com

SDC HACKATHON 2026

Technical & Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club
Submission Deadline: [Insert date]

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

1. Team & Project Overview

Team Name	VBall
Project / Solution Name	SMErgers
Name (Leader)	Vincent Chew Wen Jie
Email & Phone (Leader)	vincent0512@student.usm.my | +6011-20208512
University Affiliation(s)	Universiti Sains Malaysia
Live Prototype URL	https://sm-ergers-1.replit.app/
Code Repository URL (public / access granted to judges)	https://github.com/Anonys-sky/SMErgers

2. Architecture & Technology Stack

2.1 System Architecture Overview

Describe how the components declared in this document connect end-to-end (e.g., user interface → backend → AI model/API → database → cloud hosting). Attach a system block diagram as an appendix if available.

[Maximum 200 words]

A message from the job's WhatsApp group reaches the app through a webhook in the format of Meta's WhatsApp Business Cloud API, or a chat is imported from a WhatsApp text export. An optional local test connector can also read a group directly. A reader finds each change, its quantity and price, and whether both sides agreed, using the signed quote, rate card, company directory and past jobs. The default reader is a set of Python rules. When chosen on import, an AI reader sends the chat text to our own RunPod Serverless endpoint running Llama 3.1 8B; plain code then checks every price and status, and the rules take over if the endpoint is unreachable. Changes enter an approval workflow with project manager, project director and client roles, checked on the server. Each confirmation creates a new quote version, queues a summary message for the group and the next approver, and writes a hash-chained entry to the activity log. Outputs are a draft bill, a CSV, a read-only JSON API and the log. The interface is server-rendered Jinja pages on Flask, hosted on Replit. Data is held in memory, so a restart resets the demo.

2.2 Languages, Frameworks & Libraries

List the main programming languages, frameworks, and third-party libraries that your team did not create. Open-source use is permitted provided it is declared and properly licensed/attributed.

Name & Version	Purpose in Project	License
Python 3.12	Application language	PSF License
Flask 3.1.3	Web framework and routes	BSD-3-Clause
Jinja2, MarkupSafe, Werkzeug (installed with Flask)	Templates, safe escaping of chat text, request handling	BSD-3-Clause
tzdata 2026.5	Time zones for message times	Apache-2.0
pytest 9.1.1	Automated tests (290+)	MIT
Llama 3.1 8B Instruct, served by vLLM on RunPod Serverless (Ollama for local testing)	AI reader that finds changes in an imported chat; not used for approvals or pricing	Llama 3.1 Community License (model); Apache-2.0 (vLLM); MIT (Ollama)
Node.js ≥ 18 with whatsapp-web.js ^1.34.7 (and its Puppeteer dependency)	Optional local WhatsApp test connector that reads a test group directly; unofficial, not part of the product	Apache-2.0
qrcode (npm) ^1.5.4	QR code on the connector's pairing page	MIT
HTML, CSS, a few lines of JavaScript	User interface	none

3. APIs & Cloud Services

3.1 External APIs

List every external API your solution calls (GenAI, maps, payments, messaging, data feeds, etc.). Do not include API keys or secrets in this document.

API & Provider	Purpose in Project	Access Tier (free / paid / sponsor-provided)
WhatsApp Business Cloud API (Meta)	Design and webhook format for receiving group messages and delivering updates. **Not called live in the prototype**: messages are synthetic and outgoing messages are queued and shown on screen.	Not used live; production use needs a verified business account
RunPod Serverless API (OpenAI-compatible)	The AI reader sends an imported chat's text to our own RunPod endpoint; key kept in Replit Secrets	Pay per second, from the US$25 sponsor credit
WhatsApp Web, through the unofficial whatsapp-web.js library	Optional local test connector: reads a synthetic test group directly on a spare test number. Not an official API and not part of the product	None; may breach WhatsApp's terms, so test numbers only
None other	No hosted GenAI API, maps, payments or data feeds are called	n/a

3.2 Cloud Platforms & Deployment

List every cloud service used for hosting, storage, databases, authentication, or compute, and indicate how the prototype is deployed.

Service & Provider	Purpose in Project	Plan / Region
Replit	Development, hosting and the live demo deployment	Replit Core, provided to participants by SDC.
RunPod (Serverless)	GPU inference for the AI reader: Llama 3.1 8B Instruct on a vLLM worker (runpod/worker-v1-vllm)	Serverless endpoint 2y90x5ojwpgxnh, one RTX 4090 (24 GB) worker, scales to zero
GitHub	Source code, commit history	Free, private while building, public before submission

4. AI Tools, Models & Datasets

4.1 AI Models Used in the Solution

List every AI model embedded in or called by your application, including pre-trained, fine-tuned, and self-trained models.

Model & Version	Source / Provider	Usage (pre-trained / fine-tuned / self-trained)	License / Terms
Llama 3.1 8B	Meta	run on our own RunPod Serverless endpoint (vLLM), as the AI reader that finds changes in an imported or directly read chat; scored on the blind chats through that RunPod endpoint (an RTX 4090), with earlier runs through Ollama on a laptop	https://dev.meta.ai/llama/llama3/license

Only Llama 3.1 8B runs inside the application. Claude (Anthropic) and GPT-6 Sol (OpenAI) were used as development tools to help build the project; they are declared in section 4.3 and do not run inside the application.

4.2 Datasets

List all datasets used for training, fine-tuning, evaluation, or in the running application. State clearly if a dataset contains personal or sensitive data.

Dataset Name	**Source / URL**	**Type (public / private / self-collected / synthetic)**	**License**	**Personal Data? (Y/N)**
Synthetic event chats (8, three of them blind test chats written by separate AI agents), quotes (7), vendor invoice, answer keys	Written by the team, in data/ of the repository	synthetic	Apache-2.0 (repository)	N
Synthetic job history (10 past jobs)	Written by the team, data/history/past_jobs.json	synthetic	Apache-2.0	N
Synthetic company directory	Written by the team, data/team.json	synthetic	Apache-2.0	N

4.3 AI-Assisted Development Tools

Tools used to help write code, design, content, or documentation (e.g., chat assistants, code completion tools, image generators). Declaring these tools does not reduce your score; undeclared use may.

Did your team use any generative-AI tool to help create code, design, text, or other project material?

☑ **Yes** ☐ No

If yes, list them below:

Tool & Version	**Used For**	**Extent of Use (minor / moderate / major)**
Claude (Claude Code; Opus 5.5 and Sonnet 5.5), Anthropic	Writing and refactoring code, tests, documentation, the pitch deck and the proposal text; research and analysis	Major
Replit Agent, Replit	Run configuration, deployment publishing, startup scripts and tests	Moderate

Briefly describe how your team reviewed, tested, and took responsibility for AI-generated output:

[Maximum 100 words]

The problem comes from a teammate's experience at an event company; the team chose and locked it, and made the design decisions. AI wrote and refactored code, which we reviewed, edited and tested: every change was committed in small steps, and 290+ automated tests passed. We checked the running pages by hand for each role. We scored the extractor against answer keys and recorded in docs/results.md that the first score is not an accuracy figure. Every team member can explain how chats become changes, how prices are chosen, how approvals are enforced and how the log detects edits.

4.3 (continued): additional AI-assisted development tool

GPT-6 Sol, OpenAI	User-interface design and front-end styling	Moderate

5. Sponsor-Provided Resources

Declare any official SDC sponsor-provided GenAI API, cloud credits, tools, mentoring, or data that your team used.

Did your project use any official SDC sponsor-provided resource?

☑ **Yes** ☐ No

If yes, list them below:

Sponsor	**Resource (API / credits / platform / data)**	**How It Was Used in the Project**
SDC / XMUM AI Club (platform partner: Replit)	Replit Core subscription for each participant	Development environment and hosting of the live prototype
SDC / RunPod	US$25 team credit	**Used:** Serverless vLLM endpoint 2y90x5ojwpgxnh (RTX 4090, 24 GB) runs the AI reader's model, Llama 3.1 8B Instruct. Dashboard screenshots: Appendix A.

Was the sponsor resource essential to the core functionality of the prototype, or could it be replaced? Explain briefly:

[Maximum 100 words]

Replit is required by the hackathon rules and is where the prototype is built and hosted. The application itself is plain Python and Flask, so it could run on any host that runs Python, such as another cloud service or a local server. Nothing in the code depends on a Replit-specific service, apart from its run configuration file. RunPod is used for the one GPU task, running the AI reader's model (inference). It is not used for training, because we train no model.

6. Security, Privacy & Responsible Use

6.1 Data Handling & Privacy

What user or personal data does your application collect, store, or send to third-party services (including AI APIs)? How is it protected, and how long is it kept? State "No user data is collected" if applicable.

[Maximum 150 words]

The prototype collects no real user data. All chats, quotes, companies and people are invented and labelled "Simulated". Data is held in memory only and is erased when the app restarts or when "Reset demo data" is pressed. A session cookie stores which demo role is being viewed. Chat text leaves the app only when the AI reader is chosen, and then goes to our own RunPod endpoint; no API keys are in the repository. In a real deployment, chat exports contain personal data of clients and vendors, so we would tell them why the group is read, keep only the messages linked to changes, delete the rest after import, limit access by role, set a retention period and honour access and deletion requests, following PDPA principles. A pilot with real chats needs a legal review first.

6.2 Known Limitations & Risks

Identify the main technical limitations of your AI components (e.g., hallucination, bias, rate limits, dependency on a single provider) and any mitigation you have implemented.

[Maximum 150 words]

The AI reader can misread a chat: on three blind synthetic chats it fully got 25 of 29 changes right (the rules got 17; 21 after fixing its errors, not independent), so it can mark an unconfirmed change as ready. Plain code rejects any price not written in the chat or the quote, and nothing reaches the bill without a person's check and the client's confirmation; missing prices and discounts always go to a person. The 29-change sample is small and synthetic, so we claim no real-world accuracy (docs/results.md). Meta's Groups API allows only 8 people per group and needs a verified business, so the prototype uses synthetic webhook payloads and chat export. Data is in memory; sign-in is simulated. An unofficial WhatsApp connector, used for team testing and labelled as such in our demo, is not part of the product and may breach WhatsApp's terms.

7. Compliance & Readiness Checklist

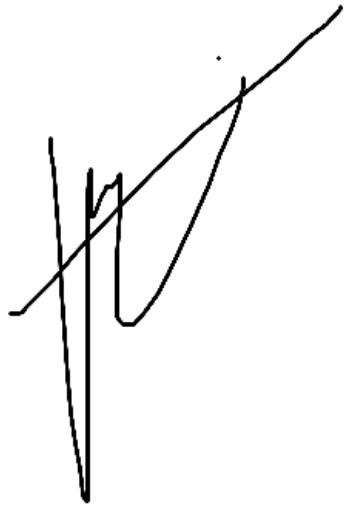
Tick all boxes that apply before submitting.

- ☑ All APIs, cloud services, AI tools, models, and datasets used in the project are declared in this document.
- ☑ All third-party code, datasets, and models are used in accordance with their licenses, and attribution is provided where required.
- ☑ No API keys, passwords, or other credentials are exposed in the code repository, prototype, or this document.
- ☑ Any use of generative-AI tools for development has been disclosed and reviewed by the team.
- ☑ Any use of official SDC sponsor resources has been declared in Section 5.
- ☑ The prototype described here is accessible to judges through the links provided in Section 1.
- ☑ All fields in this template have been completed within the word limits stated.

8. Declaration

By submitting this document, the team leader confirms that the information provided is complete and accurate to the best of the team's knowledge, and that every API, cloud service, AI tool, dataset, model, and sponsor resource used in the project has been declared.

[Signature]

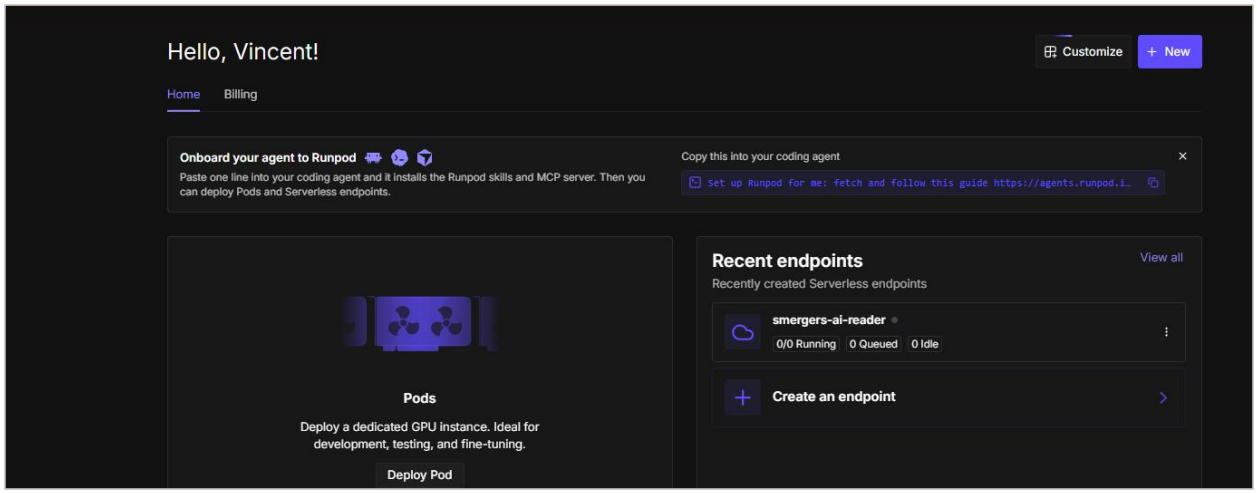


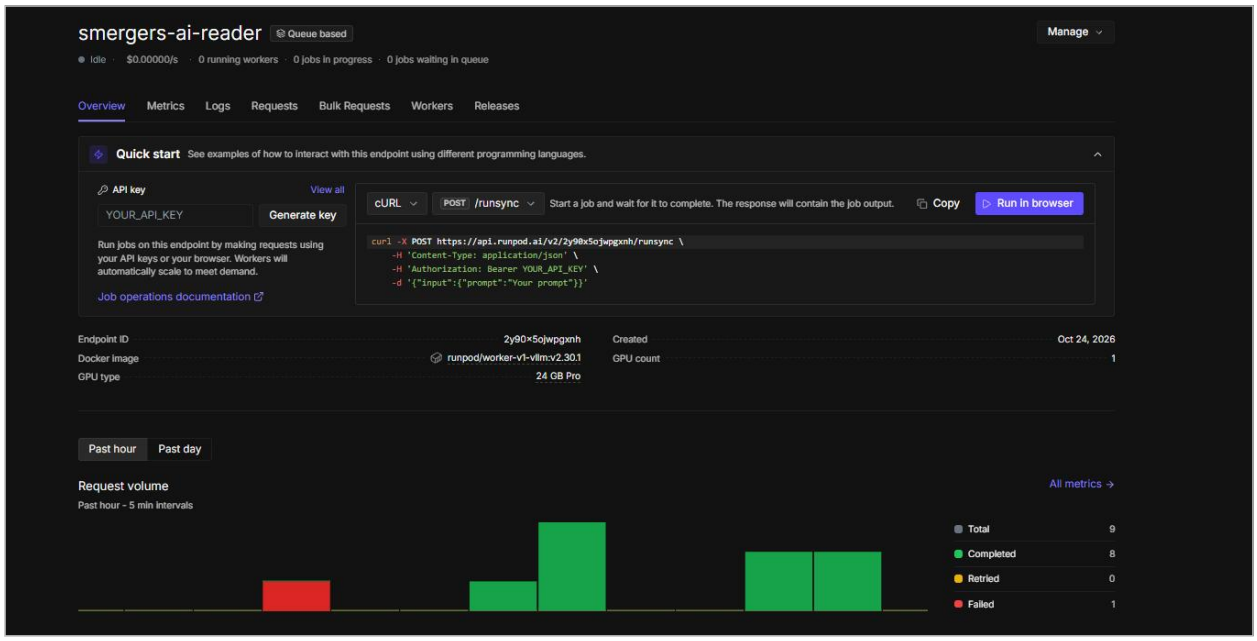
[Date]

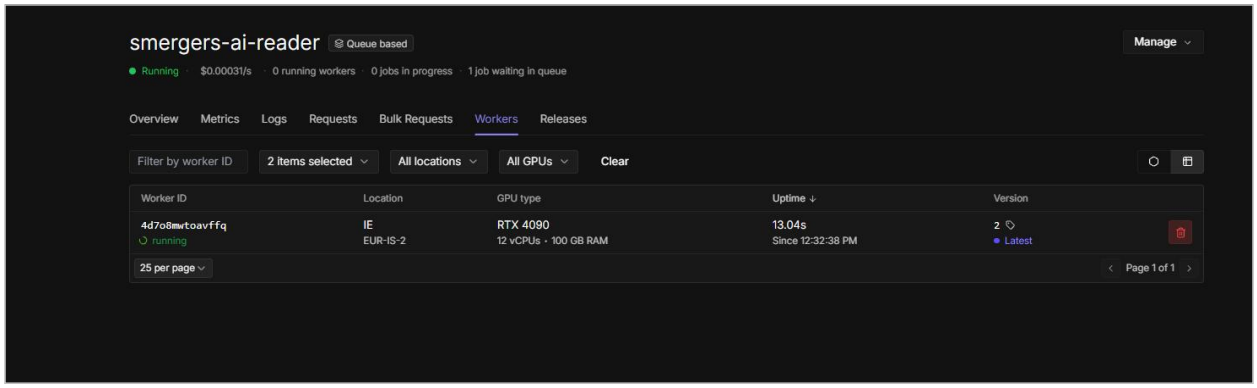
24th October 2026

Appendix A: RunPod proof of use (handbook 10.7)

Team VBall's own RunPod account: Serverless endpoint 2y90x5ojwpgxnh (URL https://api.runpod.ai/v2/2y90x5ojwpgxnh), GPU type RTX 4090 (24 GB), worker runtime and usage, as shown below.







Appendix B: Replit proof of use (handbook 10.6)

Replit proof of use

Team VBall | SMErgers | SDC Hackathon 2026, Domain 2

---

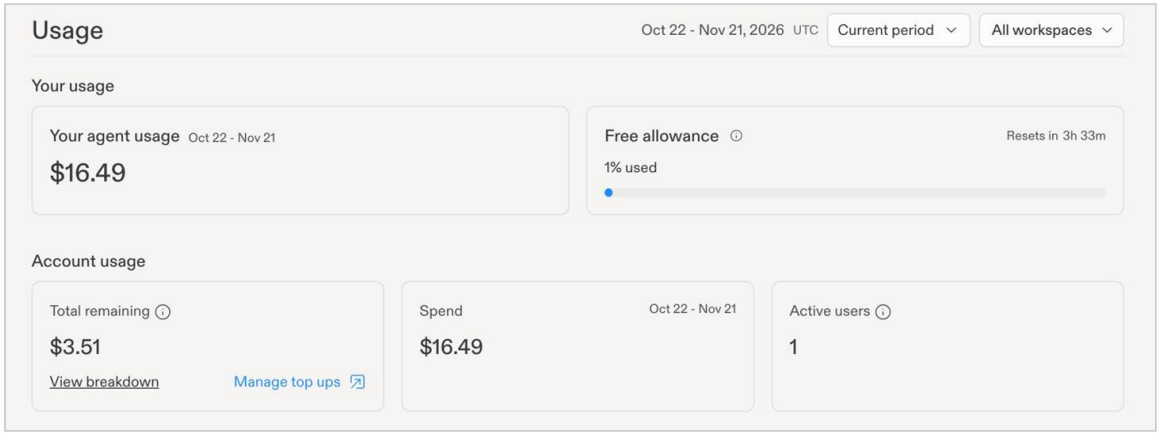
1. Live deployment on Replit

https://sm-ergers-1.replit.app

The deployed SMErgers app (Python and Flask), running on Replit Deployments.

2. Replit usage page (team member's account)

Period Oct 22 to Nov 21, 2026: agent usage $16.49, one active user.



Replit is where the team designs, tests and deploys SMErgers.

Code: https://github.com/Anonys-sky/SMErgers